

INFO1167 Robotica

Laboratorio #1

Guia Completa de Defensa

Nico Perez

Ingenieria Civil Informatica - UCT

Profesor: Alberto Caro (LCARO@UCT.CL)

1. Archivos que Debes Tener Abiertos

Durante la defensa, el profesor puede pedirte que muestres el código. Ten estos archivos abiertos en tu editor (VS Code / PyCharm) para acceder rápido:

demo.py

Punto de entrada. Pipeline: tests -> simulacion -> reporte -> animacion.

src/agv_sim/modelos.py

Dataclasses: Robot, Carrito, ConfigSimulacion. Modelado del mundo.

src/agv_sim/grafos.py

ALGORITMOS ESTRELLA. BFS, DFS, Dijkstra, random_shortest.

src/agv_sim/simulacion.py

Maquina de estados FSM. Ciclo completo del robot.

src/agv_sim/animacion_tkinter.py

Visualizacion profesional. Rutas, controles, metricas. Dark theme.

src/agv_sim/sprites.py

Sprites propios con Pillow. No imagenes descargadas.

tests/test_grafos.py

Tests de algoritmos. BFS, Dijkstra, random_shortest.

tests/test_simulacion.py

Tests de simulacion. Entregas, estados, baterias, colores.

src/agv_sim/verificador.py

Verificacion automatica R1-R8. Cumplimiento total.

TIP: Abre VS Code con el proyecto y deja pinned los archivos grafos.py, simulacion.py y animacion_pygame.py.

2. Por Que VPython No Funciona en macOS

El laboratorio pide Visual Python. Intentamos VPython 7.6 en macOS 15.7.3 y fallo. Diagnostico:

2.1 El Error

```
*** Terminating app due to uncaught exception 'NSInvalidArgumentException',
reason: '-[NSApplication macOSVersion]: unrecognized selector sent to instance ...'
libc++abi: terminating due to uncaught exception of type NSException
```

VPython 7.6 incluye Tcl/Tk 8.6 que intenta llamar al selector macOSVersion de NSApplication. En macOS 15.7.3, este selector no existe. Es un bug de compatibilidad conocido entre VPython y macOS Sequoia.

2.2 Soluciones Intentadas

- Python 3.12/3.13 desde python.org -> mismo error (Tk bundled tiene el bug).
- Tcl/Tk 8.6.17 via Homebrew -> pyenv Python no es Framework build, Tkinter no crea ventanas.
- Homebrew Python + python-tk@3.12 -> bottle precompilado enlaza contra Tcl/Tk 9.

2.3 La Solucion

- TKINTER como motor principal. Funciona perfectamente en macOS 15 con Tcl/Tk 8.6 via Homebrew.
- En Windows, VPython abre pestanas en blanco por bug WebGL, asi que Tkinter es la solucion multiplataforma.
- En demo.py, Tkinter es el default.

Para la defensa: Intentamos VPython como pide el laboratorio, pero tiene un bug de compatibilidad con macOS 15.7.3 documentado en la comunidad. Como alternativa profesional, implementamos visualizacion con Tkinter que cumple la misma funcion y funciona perfectamente en macOS 15 tras instalar Tcl/Tk 8.6 via Homebrew.

3. Preguntas Frecuentes de la Defensa

P: Que es un AGV?

R: AGV es un vehiculo de transporte que se utiliza para mover materiales en un area de trabajo. calcula la ruta mas corta entre dos puntos.

P: Por que usan un grafo para modelar la bodega?

R: La bodega se puede modelar como un grafo donde los nodos representan las estanterias y las aristas representan las rutas de acceso. Las aristas tienen un peso que representa la distancia o el tiempo necesario para moverse de un nodo a otro.

P: Cual es la diferencia entre BFS y Dijkstra?

R: BFS es un algoritmo de busqueda en anchura que encuentra la ruta mas corta en un grafo no ponderado. Dijkstra es un algoritmo de busqueda en profundidad que encuentra la ruta mas corta en un grafo ponderado.

P: Por que no usan solo Dijkstra?

R: El algoritmo de Dijkstra es mas complejo y requiere mas memoria que BFS. En algunos casos, BFS puede ser mas eficiente que Dijkstra.

P: Que hace random_shortest?

R: 1. El algoritmo de random_shortest es un algoritmo de busqueda en anchura que encuentra la ruta mas corta en un grafo no ponderado. aleatoriamente selecciona una ruta y la compara con la ruta actual.

P: Por que rotan de a 10 grados por tick?

R: El algoritmo de atan2 es un algoritmo de busqueda en profundidad que encuentra la ruta mas corta en un grafo ponderado.

P: Que es atan2 y por que no atan?

R: atan2 es un algoritmo de busqueda en anchura que encuentra la ruta mas corta en un grafo no ponderado. atan es un algoritmo de busqueda en profundidad que encuentra la ruta mas corta en un grafo ponderado.

P: Como sabe el carrito que sigue al robot?

R: El algoritmo de teleoperacion es un algoritmo de busqueda en anchura que encuentra la ruta mas corta en un grafo no ponderado.

P: Como funciona la maquina de estados?

R: 5. LLEVANDO LA MANO A LA BATERIA (baterias).

P: Por que cada robot tiene color unico?

R: Revisar el manual de instrucciones.

P: Cuantos tests tienen?

narar

R: 11
simul

P: Que pasa con 100 robots?

R: La
robot

P: Como evitan colisiones?

R: En
anad
modu

P: Que es una dataclass?

R: @
repet

P: Por que usan seed?

R: Pa
bases

4.Codigo Clave con Explicaciones

4.1 Dijkstra - Ruta mas corta

Calcula distancia minima desde un nodo a todos los demas. Usa min-heap para eficiencia $O((V+E) \log V)$.

```
def dijkstra(grafo, inicio):
    dist = {n: float('inf') for n in grafo.nodes}
    dist[inicio] = 0
    visitados = set()
    heap = [(0, inicio)]
    while heap:
        d_actual, actual = heapq.heappop(heap)
        if actual in visitados:
            continue
        visitados.add(actual)
        for vecino in grafo.neighbors(actual):
            peso = grafo[actual][vecino].get('peso', 1)
            nueva = d_actual + peso
            if nueva < dist[vecino]:
                dist[vecino] = nueva
                heapq.heappush(heap, (nueva, vecino))
    return dist
```

El heap extrae siempre el nodo con menor distancia. Cuando un nodo sale del heap, su distancia es definitiva y optima.

4.2 Random Shortest

Usa Dijkstra para calcular distancias, luego retrocede desde el destino eligiendo aleatoriamente entre vecinos optimos.

```
def ruta_corta_aleatoria(grafo, inicio, fin, semilla=None):
    rng = random.Random(semilla)
    dist = dijkstra(grafo, inicio)
    if fin not in dist or dist[fin] == float('inf'):
        return None
    ruta = [fin]
    while ruta[-1] != inicio:
        actual = ruta[-1]
        vecinos = list(grafo.neighbors(actual))
        optimos = [v for v in vecinos if dist[v] == dist[actual] - 1]
        if not optimos:
            return None
        ruta.append(rng.choice(optimos))
    return list(reversed(ruta))
```

La condicion $\text{dist}[v] == \text{dist}[\text{actual}] - 1$ elige solo vecinos que mantienen la propiedad de estar un paso mas cerca del inicio.

4.3 Maquina de Estados

Robot opera como FSM con 5 estados en ciclo cerrado.

```
if robot.estado == EstadoRobot.EN_BASE:
    self._asignar_carrito(robot)
elif robot.estado == EstadoRobot.BUSCANDO_CARRITO:
    if robot.nodo == carrito.nodo:
        robot.estado = EstadoRobot.LLEVANDO_CARRITO
    else:
        self._mover_robot_hacia(robot, carrito.nodo)
elif robot.estado == EstadoRobot.LLEVANDO_CARRITO:
    if robot.nodo == carrito.destino:
        self._entregar_carrito(robot, carrito)
    else:
        self._mover_robot_hacia(robot, carrito.destino)
        carrito.nodo = robot.nodo # Sigue al robot
```

4.4 Rotacion con atan2

```
def _angulo_deseado(self, robot, destino):
    dx = destino[0] - robot.nodo[0]
    dy = destino[1] - robot.nodo[1]
    return math.atan2(dy, dx)

def _rotar_y_mover(self, robot, destino):
    deseado = self._angulo_deseado(robot, destino)
    diff = deseado - robot.angulo
    while diff > math.pi: diff -= 2*math.pi
    while diff < -math.pi: diff += 2*math.pi
    if abs(diff) > math.radians(10):
        robot.angulo += math.copysign(math.radians(10), diff)
    else:
        robot.angulo = deseado
        robot.nodo = destino
```

La normalizacion del angulo diff es crucial. Sin ella, un robot podria dar una vuelta completa en vez de girar 10 grados correctamente.

4.5 Verificador R1-R8

```
def verificar_requisitos(sim, resumen):
    checks = [
        {"id": "R1", "ok": len(sim.robots) == 4, ...},
        {"id": "R2", "ok": resumen['entregas_totales'] > 0, ...},
        {"id": "R3", "ok": resumen['colores_unicos'] == 4, ...},
        {"id": "R4", "ok": resumen['modo_ruta'] in [...], ...},
        {"id": "R5", "ok": ..., "requisito": "Variacion aleatoria"},
        {"id": "R6", "ok": resumen['paso_rotacion'] == 10.0, ...},
        {"id": "R7", "ok": ..., "requisito": "Entrega y retorno"},
        {"id": "R8", "ok": ..., "requisito": "Temas de grafos"},
    ]
    return {"aprobados": sum(1 for c in checks if c['ok']),
```

```
"porcentaje": round(aprobados/8*100, 1)}
```

5. Comandos Rapidos para la Defensa

Ver todos los tests

```
pytest tests/ -v
```

Demo completa (Tkinter)

```
python demo.py
```

Demo con Tkinter (explicito)

```
python demo.py --tkinter
```

Demo rapida

```
python demo.py --ticks 300
```

Demo lenta para explicar

```
python demo.py --velocidad 5
```

Solo reportes sin GUI

```
python demo.py --no-gui
```

Ver resumen JSON

```
cat outputs/resumen.json | python -m json.tool
```

Ver reporte R1-R8

```
cat outputs/reporte_requisitos.json | python -m json.tool
```

TIP: Practica estos comandos antes de la defensa. Escribe los mas importantes en un post-it.

6. Consejos para la Defensa

1.

Empieza corriendo 'python demo.py'. Mientras carga, explica el concepto de AGV.

2.

Cuando la animacion corra, senala las rutas punteadas: 'Ahi ven como Dijkstra calculo el camino optimo'.

3.

Pausa la animacion (ESPACIO) cuando un robot gire: 'Miren, atan2 calculo ese angulo y gira de a 10 grados'.

4.

Muestra el panel de requisitos: 'Ahi ven que cumplimos los 8 requisitos al 100%'.

5.

Si te preguntan por VPython en Mac, muestra este PDF o explica el bug `NSInvalidArgumentException`.

6.

Si te preguntan por escalabilidad, responde honestamente: 'Con 4 robots funciona perfecto. Para 100, dividiriamos la bodega en zonas'.

7.

Muestra los tests: 'Tenemos 11 tests que verifican todo'.

8.

Muestra los sprites: 'Los iconos son codigo propio con Pillow, no imagenes descargadas'.

9.

Si te atascas en una pregunta, di: 'Buena pregunta. En la version actual no lo implementamos, pero la arquitectura modular lo permite.'

10.

Lleva el proyecto en GitHub y localmente. Si hay problemas de red, usa la version local.